

人工智能原理与应用

詹紫琳

邮箱: zilin.zhan@szu.edu.cn

Spring 2026



深圳大学
SHENZHEN UNIVERSITY

回顾

- 树搜索的边缘、路径、问题解、解的代价；搜索树
 - 搜索算法主要性能评价指标: 完备性、最优性、时空复杂度
 - 搜索规则影响算法性能: 排序、选择、检查
-
- **完备性 (completeness)** : 当问题存在解的时候, 这个算法是否能保证找到最优解; 当不存在解时, 是否能保证报告失败?
 - **最优性 (cost optimality)** : 搜索将总是找到成本最低的解决方案吗?
(当动作有**成本/路径耗散 (path cost)** 时)
 - **时间复杂度 (time complexity)** : 找到解需要花费多少时间?
 - **空间复杂度 (space complexity)** : 在执行搜索的过程中需要多少内存?

回顾

- **有信息搜索 (informed search)**：它利用启发式函数 (heuristic function, $h(n)$)，去估计某个非目标状态离目标状态的“希望程度” / “代价”，从而指导搜索顺序。
- **无信息搜索 (uninformed search) / 盲目搜索 (blind search)**：它对问题的状态空间只有基本的定义（初始状态、目标测试、后继函数、路径代价），没有任何关于目标位置的额外知识。因此它不能判断一个非目标状态是否更接近目标，只能依靠系统性遍历或规则来扩展节点。

广度/宽度优先搜索 (breadth-first search, BFS)

图搜索算法的一个实例

基本思想

- 从初始状态开始，一层一层地向外扩展，**即先扩展距离初始状态最近的节点**，再扩展更远的节点。
- 使用队列（FIFO/ First In, First Out）作为存储结构，存储等待探索的状态。
 - FIFO例如：先入库的货物先出库，避免存货过期

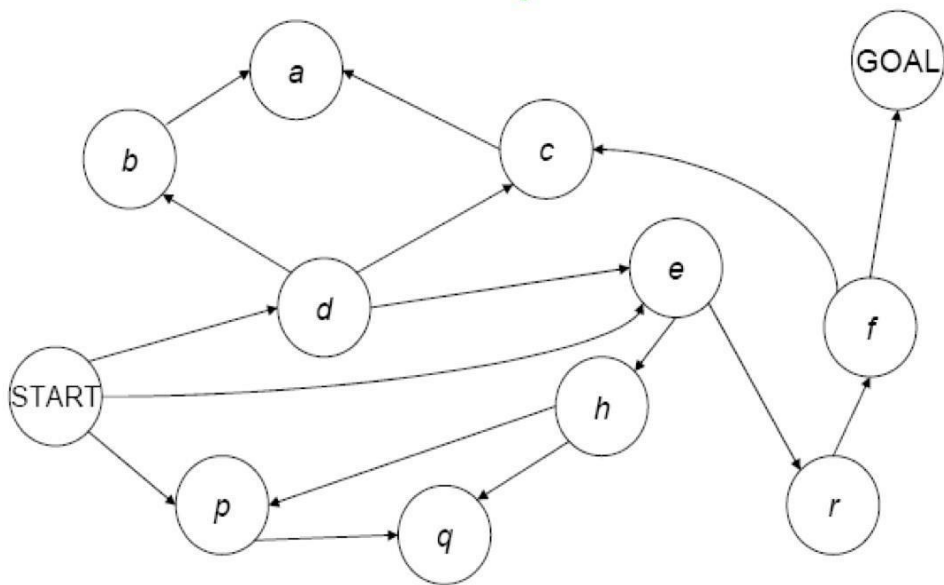
图解广度优先搜索

广度/宽度优先搜索 (breadth-first search, BFS)

步骤

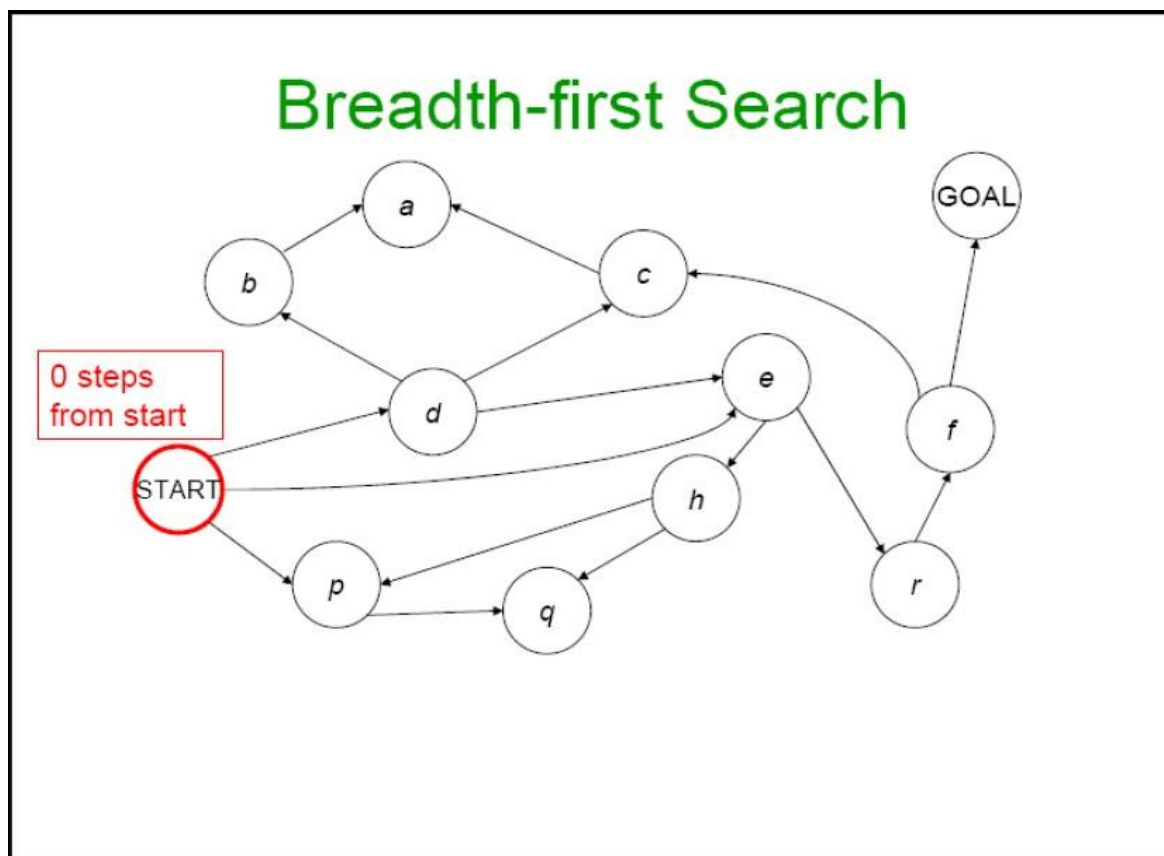
1. 将初始状态放入队列。
2. 重复以下操作，直到队列为空或找到目标：
 - 从队列头部取出一个节点。
 - 如果该节点是目标，结束搜索。
 - 否则，扩展该节点的所有后继状态，并将它们加入队列尾部。

广度/宽度优先搜索 (breadth-first search, BFS)

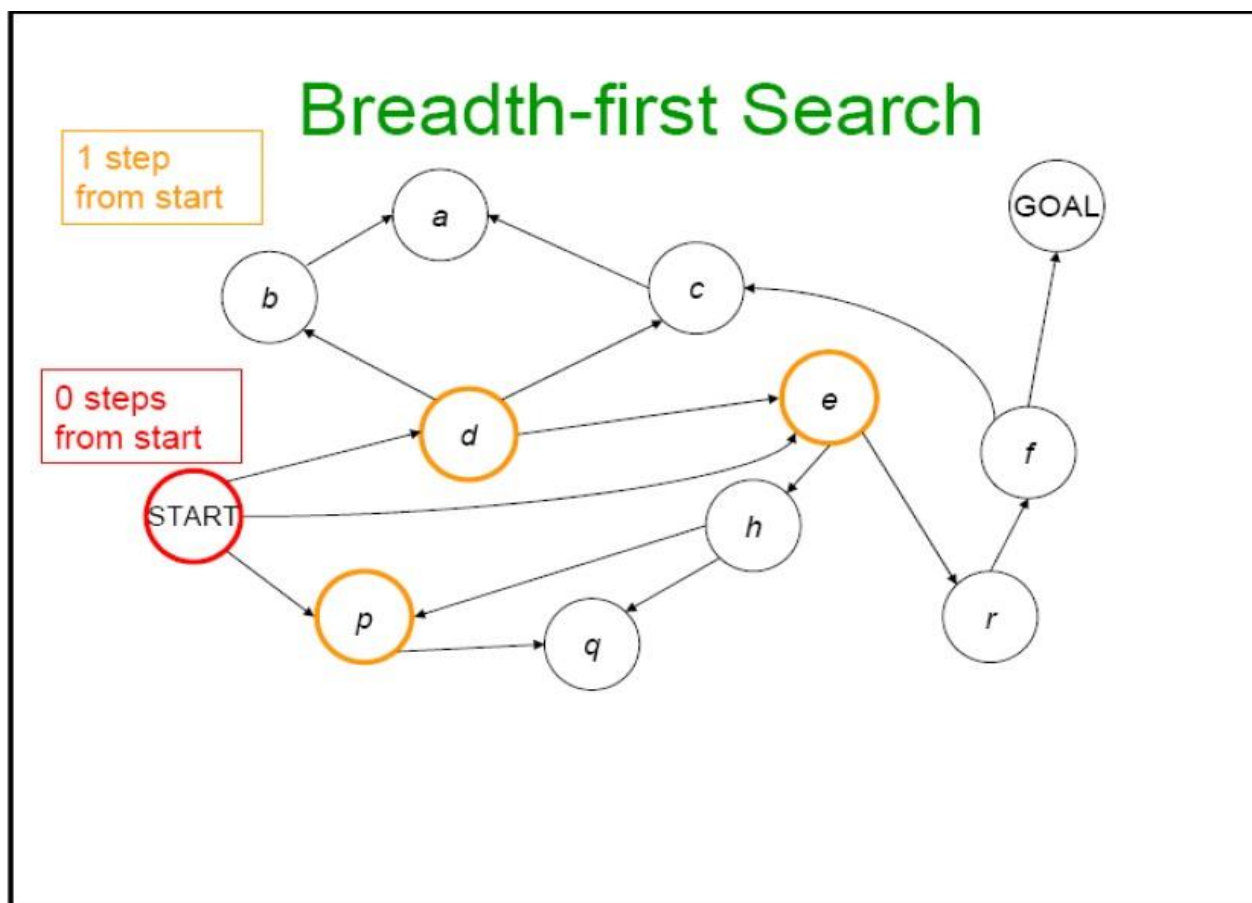


- 先把起始态放到边缘上
- 把所有能从起始态恰好一步能到达的状态都放到边缘上；把路径添加到边缘
- 把所有能从起始态恰好二步能到达的状态都放到边缘上；把路径添加到边缘
- 把所有能从起始态恰好三步能到达的状态都放到边缘上；把路径添加到边缘
-

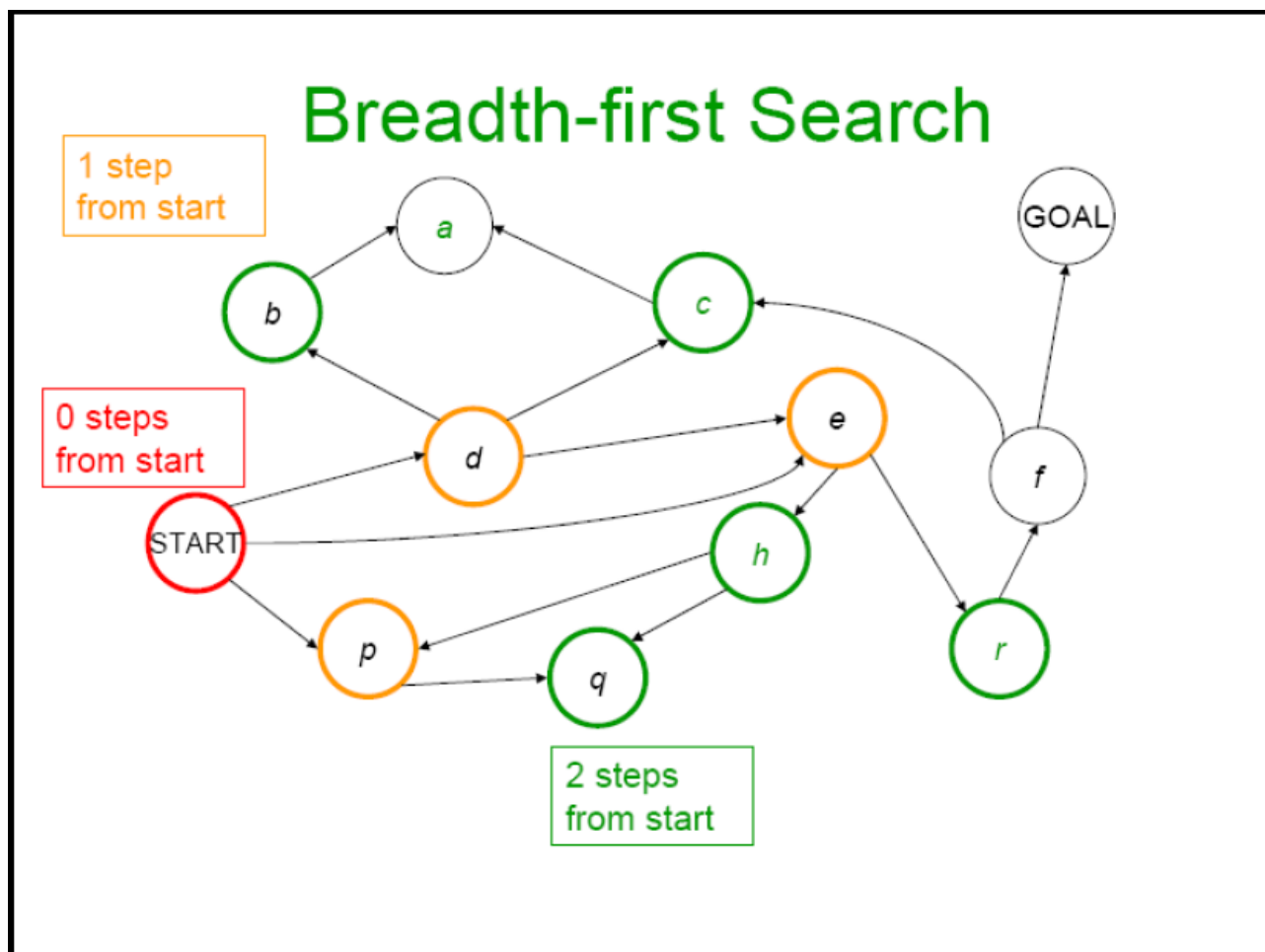
广度/宽度优先搜索 (breadth-first search, BFS)



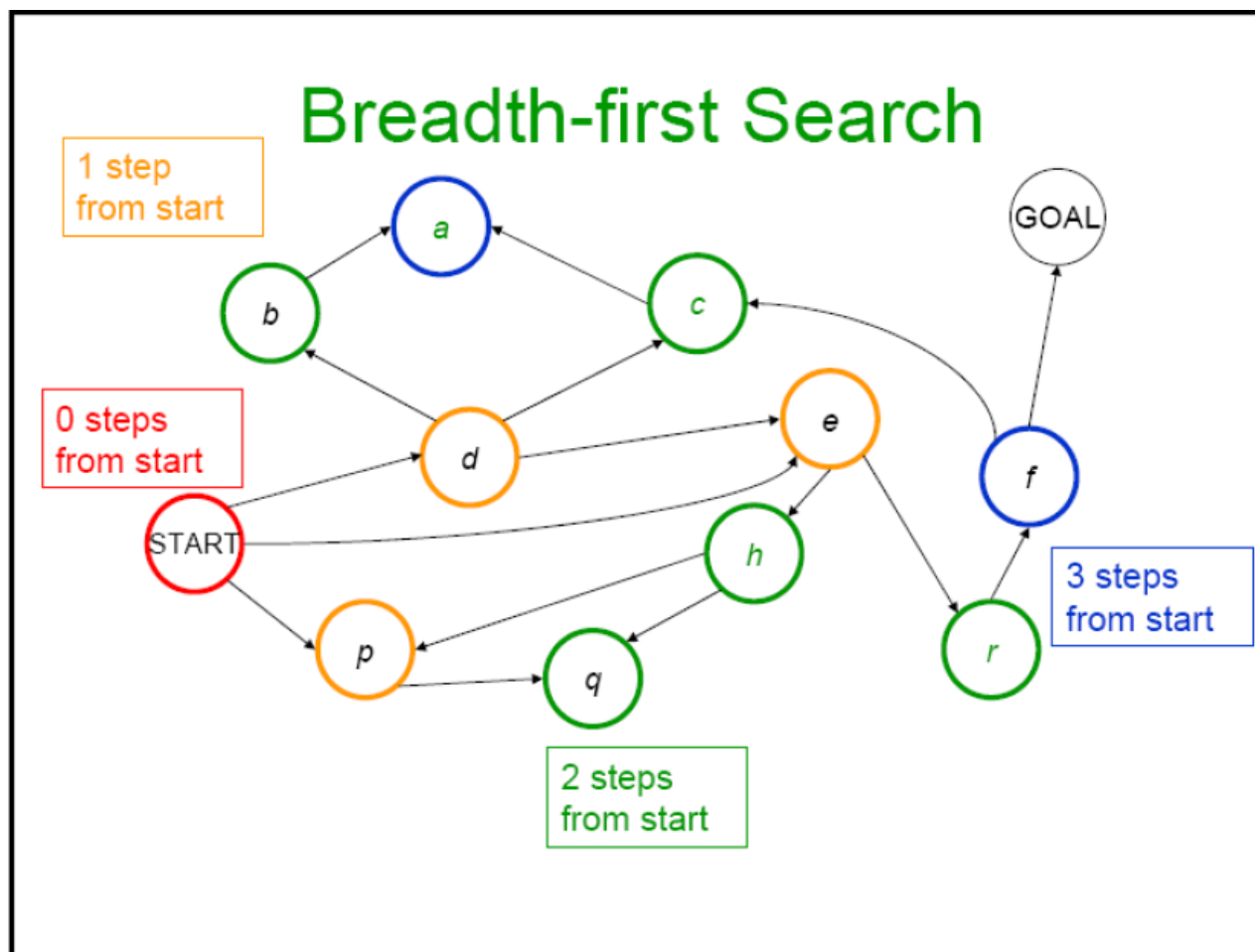
广度/宽度优先搜索 (breadth-first search, BFS)



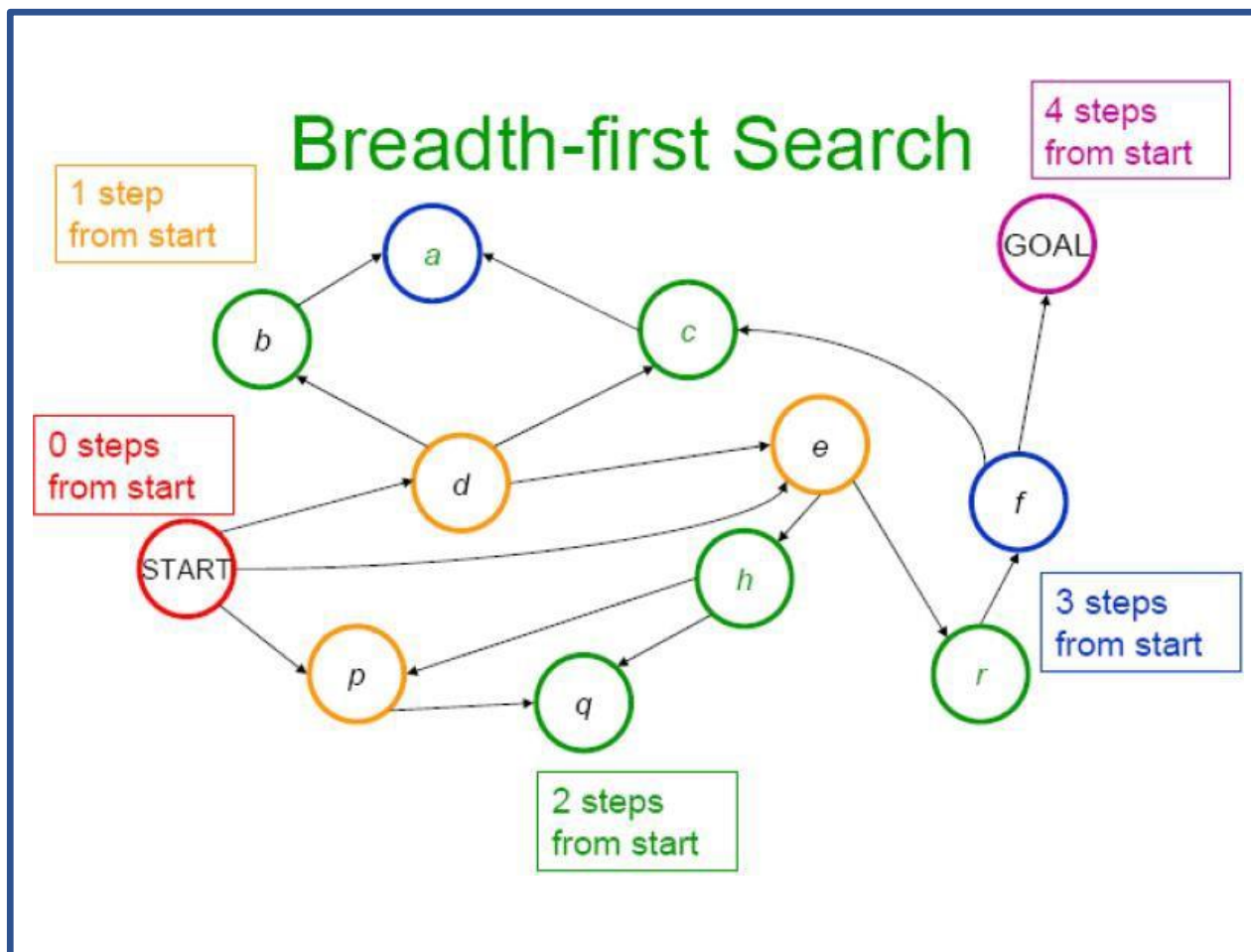
广度/宽度优先搜索 (breadth-first search, BFS)



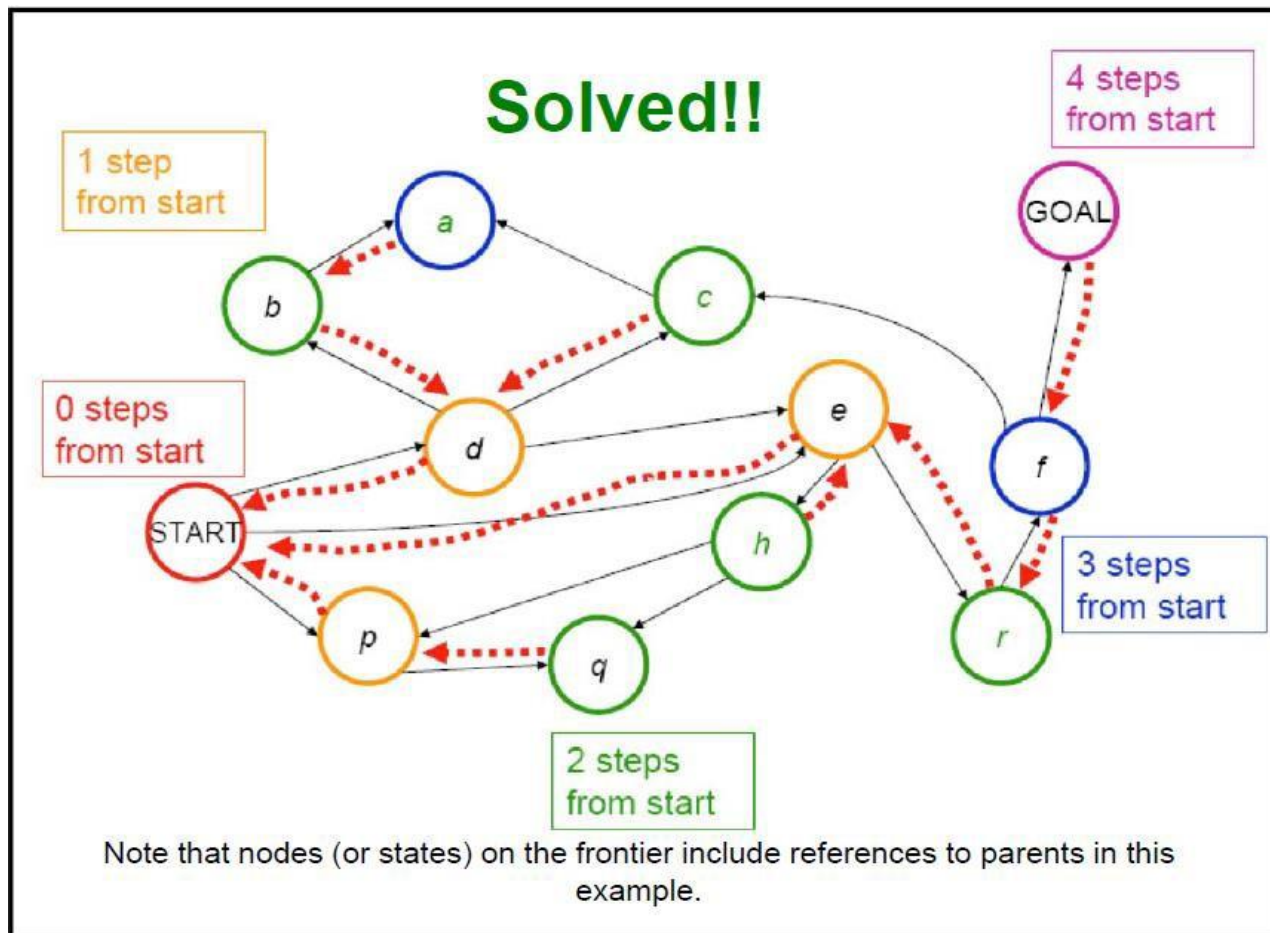
广度/宽度优先搜索 (breadth-first search, BFS)



广度/宽度优先搜索 (breadth-first search, BFS)



广度/宽度优先搜索 (breadth-first search, BFS)



请注意，在此示例中，边缘上的结点（或状态）包括对父级的引用

BFS解决水罐问题

- 初始态 = $(0,0)$, 终止态 = $(*, 2)$
- 动作 (后续函数) : 空-3-Gallon, 空-4-Gallon, 满-3-Gallon, 满-4-Gallon, 转-3-到-4, 转 4-到-3。
- 边缘 = $\{<(0,0)>\}$

BFS解决水罐问题

- 初始态 = $(0,0)$, 终止态 = $(*, 2)$
- 动作 (后续函数) : 空-3-Gallon, 空-4-Gallon, 满-3-Gallon, 满-4-Gallon, 转-3-到-4, 转 4-到-3。
- 边缘 = $\{<(0,0)>\}$
- 边缘 = $\{<(0,0),(3,0)>, <(0,0),(0,4)>\}$

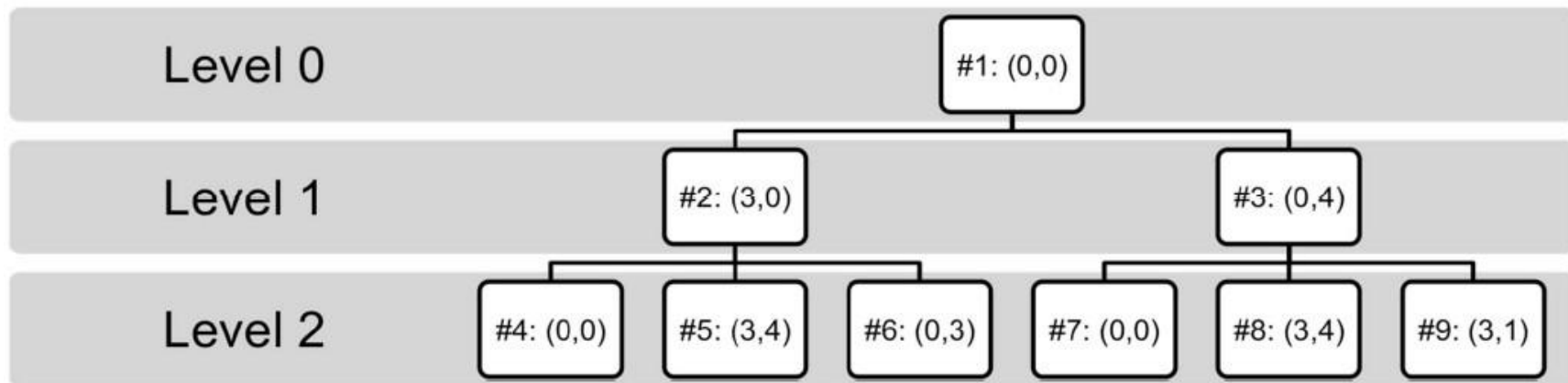
BFS解决水罐问题

- 初始态 = $(0,0)$, 终止态 = $(*, 2)$
- 动作 (后续函数) : 空-3-Gallon, 空-4-Gallon, 满-3-Gallon, 满-4-Gallon, 转-3-到-4, 转 4-到-3。
- 边缘 = $\{ \langle (0,0) \rangle \}$
- 边缘 = $\{ \langle (0,0), (3,0) \rangle, \langle (0,0), (0,4) \rangle \}$
- 边缘 = $\{ \langle (0,0), (0,4) \rangle, \langle (0,0), (3,0), (0,0) \rangle, \langle (0,0), (3,0), (3,4) \rangle, \langle (0,0), (3,0), (0,3) \rangle \}$

BFS解决水罐问题

- 初始态 = $(0,0)$, 终止态 = $(*, 2)$
- 动作 (后续函数) : 空-3-Gallon, 空-4-Gallon, 满-3-Gallon, 满-4-Gallon, 转-3-到-4, 转 4-到-3。
- 边缘/队列 = $\{<(0,0)>\}$
- 边缘/队列 = $\{<(0,0),(3,0)>, <(0,0),(0,4)>\}$ (取出 $\{<(0,0)>\}$, 扩展后继节点...)
- 边缘/队列 = $\{<(0,0),(0,4)>, <(0,0),(3,0),(0,0)>, <(0,0),(3,0),(3,4)>, <(0,0),(3,0),(0,3)>\}$
- 边缘/队列 = $\{<(0,0),(3,0),(0,0)>, <(0,0),(3,0),(3,4)>, <(0,0),(3,0),(0,3)>, <(0,0),(0,4),(0,0)>, <(0,0),(0,4),(3,4)>, <(0,0),(0,4),(3,1)>\}$

BFS解决水罐问题



广度优先搜索把状态按层排序，逐层探索状态空间。

广度优先搜索的性质

树的表达可以用来评估时空复杂度

什么是时间复杂度？衡量算法运行时需要执行多少步操作（扩展多少个结点）

- 在 BFS 中，最主要的操作就是生成和检查节点
- 所以 BFS 的时间复杂度 $\approx$ 需要扩展的节点总数

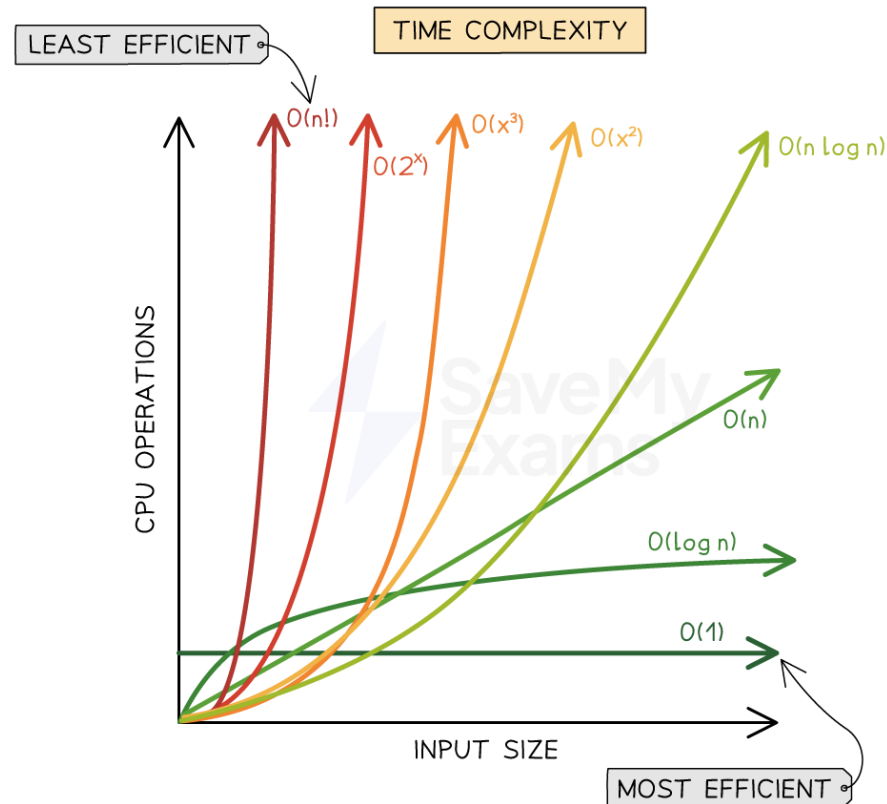
BFS 的搜索过程可以抽象为一棵 状态空间树：

- 根节点：起点 START（深度 0，路径长度 1）。
- 每个节点最多生成 b 个子节点（这里 b 称为最大分支因子）。
- 如果解的深度为 d ，那么 BFS 要把深度从 0 到 d 的所有节点都扩展出来（直到找到解），所以 $d = \text{路径长度} - 1$

Big-O表示法

广度优先搜索的性质

Big-O表示法



广度优先搜索的性质

在一棵 b 分支的树里，每个结点的最大后续状态数目（即最大分枝因子），假设 d 是解的深度：

深度 0：有 1 个节点（根）。

深度 1：最多有 b 个节点。

深度 2：最多有 b^2 个节点。

深度 3：最多有 b^3 个节点。

...

深度 d ：最多有 b^d 个节点。

因此，总节点数是一个等比数列：

$$1 + b + b^2 + b^3 + \dots + b^{d-1} + b^d = \frac{b^{d+1} - 1}{b - 1},$$

当 b 很大、 d 也很大时，总节点数近似为 b^{d+1} （呈指数级增长）

时间复杂度： $N \approx O(b^{d+1})$ / 近似写法 $O(b^d)$

广度优先搜索的性质

- 完备性：如果解存在，BFS 一定能找到。
- 最优性：在所有代价相等的情况下，BFS 保证找到最短解。
 - 先检查深度 0（起点），再检查深度 1（一步之遥的状态），再是深度 2……直到深度 d 。
 - 一旦 BFS 找到目标节点，就保证这个目标是到达目标的最短路径（以步数为代价）。
 - 如果 每一步代价不同（例如走一步花 1 元，走另一条捷径可能花 5 元），BFS 找到的只是步数最少的路径，而不一定是代价最小的路径。
- 时间复杂度： $O(b^d)$ ，其中 b 是分支因子， d 是最浅目标深度。
- 空间复杂度：同样为 $O(b^d)$ ，因为需要存储所有层的节点。

BFS 的现实问题：空间复杂性

- 举例来说：结点分支平均10，一个平均每秒可以扩展十万结点的搜索树，如果每个结点要求100字节的话：

深度	结点	时间	空间
1	1	0.01 毫秒	100 bytes
6	10 ⁶	10 秒	100 MB
8	10 ⁸	17 分钟	10 GB
9	10 ⁹	3 小时	100 GB

在大多数应用程序中，通常在我们用完时间之前就用完了空间。

深度优先搜索 (depth-first search)

基本思想

- 从初始状态开始，尽可能沿着一条路径向下走到底（到达目标或死路），再回溯到上一个分支点继续探索。
- 使用栈（LIFO/ Last In, First Out）作为存储结构，或者递归实现（最后放进去的元素，最先被取出来）。

步骤

- 将初始状态放入栈。
- 重复以下操作，直到栈为空或找到目标：
- 从栈顶取出一个节点。
- 如果该节点是目标，结束搜索。
- 否则，将它的后继状态依次压入栈顶。

图解深度优先搜索

深度优先搜索 (depth-first search)

- 初始态 = $(0,0)$, 终止态 = $(*, 2)$, 动作 (后续函数) : 空-3- Gallon, 空-4-Gallon, 满-3-Gallon, 满-4-Gallon, 转-3-到-4, 转 4- 到-3
- 边缘 = $\{<(0,0)>\}$
- 边缘 = $\{<(0,0), (3,0)>, <(0,0), (0,4)>\}$
- 边缘 = $\{<(0,0), (3,0), (0,0)>, <(0,0), (3,0), (3,4)>, <(0,0), (3,0), (0,3)>, <(0,4), (0,0)>\}$
- 边缘 = $\{<(0,0), (3,0), (0,0), (3,0)>, <(0,0), (3,0), (0,0), (0,4)>, <(0,0), (3,0), (3,4)>, <(0,0), (3,0), (0,3)>, <(0,0), (0,4)>\}$

深度优先搜索 (depth-first search)

Level 0

#1: (0,0)

Level 1

#2: (3,0)

(0,4)

Level 2

#3: (0,0)

(3,4)

(0,3)

Level 3

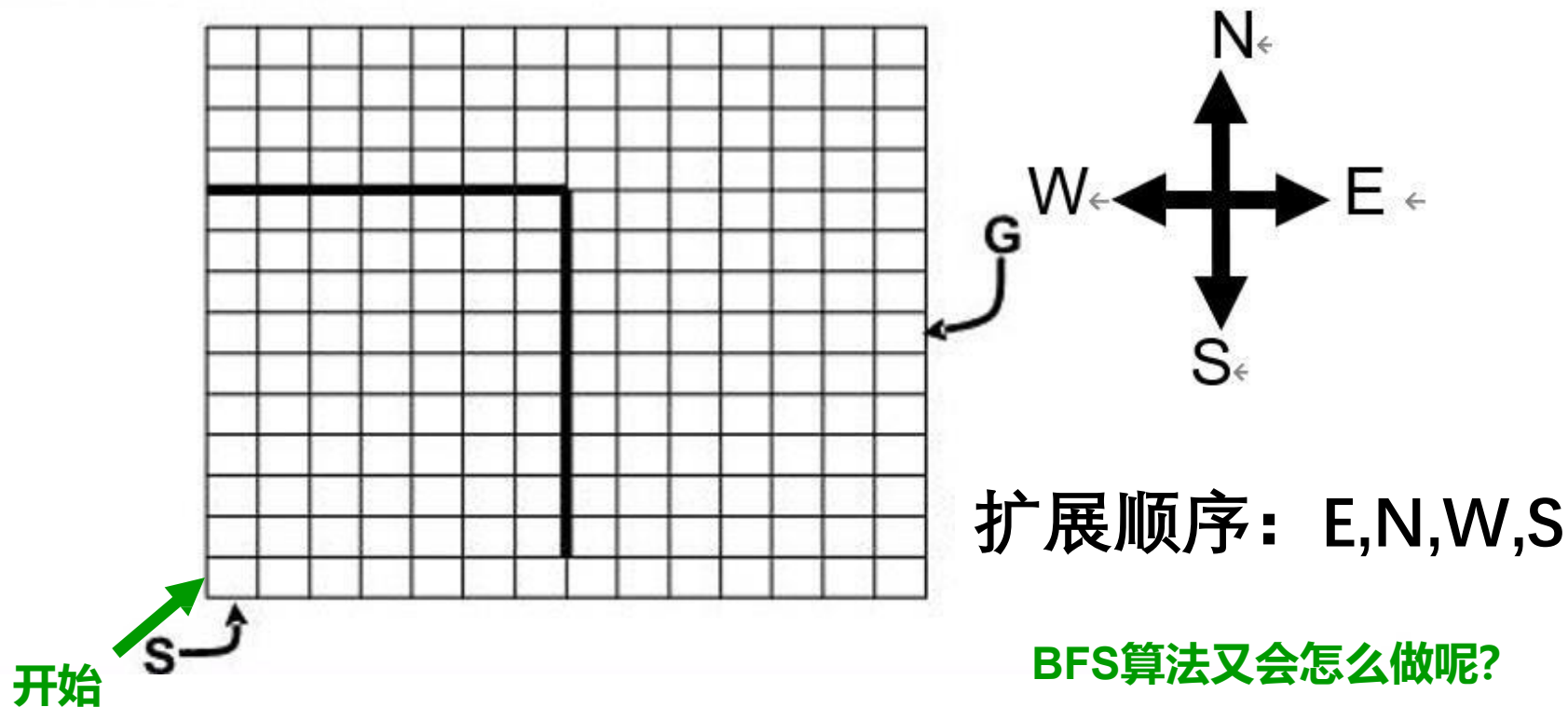
#4: (3,0)

(0,4)

红色结点是回溯点（这些结点保留在边缘上）。

示例：网格世界

- 假设状态是网格中的一个格子，你能东北西南移动，那么DFS算法怎么操作呢，假设你每次都先往东，然后再北，西，南。



性质

- 算法是完备的吗？

性质

- 算法是完备的么？

如果问题的状态空间是有限的（节点数有限），那么 DFS 最终一定会把所有节点都遍历一遍。

即使它可能绕来绕去，但因为状态总数有限，它总会“回到头”，最终能找到目标。

如果状态空间是无限深的树：

DFS 会盯住某一条路径一直往下走，可能永远走不出来。

这样，即使解在另一条浅层路径上，DFS 也可能永远找不到。

所以在无限深度的状态空间中，DFS 不保证完备。

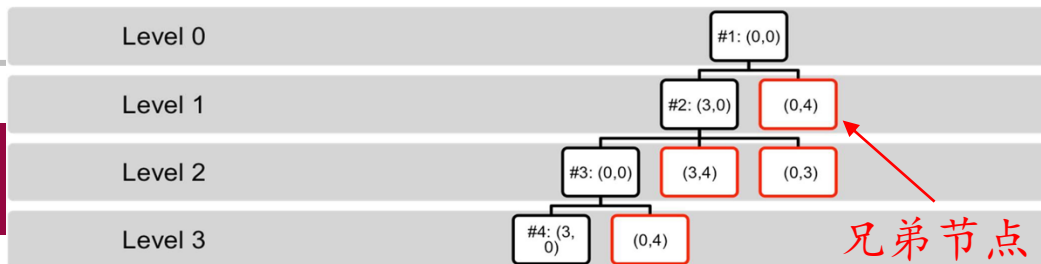
- 最优性？

不保证最优解，因为 DFS 可能先找到一条较深但代价高的路径。

性质

- 时间复杂度？
 - $O(b^m)$ ， m 是最长路径的长度
<类似BFS的计算方法，最坏情况下要访问所有节点>
- DFS v.s. BFS
 - 当 $m \gg d$ 时，DFS可能比BFS慢得多，因为它可能探索到无关的深层节点。
 - 如果解分布得比较密集，DFS有机会比BFS更快找到一个解，因为它不必像 BFS 那样逐层全面展开。

性质



● 空间复杂度？

□ $O(bm)$, 线性的! —— DFS 的数据结构通常是一个栈 (DFS的显著优势)

- 一次只探索一条路径。
- 在最坏情况下，它需要记录：
 - 路径上的所有结点：深度最多 m ，所以至少要存储 $O(m)$ 。
 - 每一层的兄弟结点（尚未展开的后继）：
 - 在深度为 0 的根节点，可能有 $b-1$ 个兄弟未展开；
 - 在深度为 1 的节点，同样可能有 $b-1$ 个兄弟未展开；
 - 以此类推，深度为 m 时仍可能有兄弟节点等待回溯。
 - 所以在最坏情况下，除了当前路径长度 m ，还要额外存储大约 $b-1$ 个兄弟 $\times m$ 层。

深度受限搜索

DFS 的问题:

- 在无限状态空间里, DFS 可能沿着一条路径无限深入, 永远不回头 → 找不到解。
- 在有限但很深的空间里, DFS 可能先找到一个很长的解, 而忽视更短的解。

解决办法:

- 设定一个最大搜索深度 d 。
- 超过深度 d 的路径就不再展开。
- 这样可以避免“路径太长”或“无限下探”的问题。

➔ 使用**深度受限搜索**!

深度受限搜索

深度受限搜索（DLS）的规则

- 从根节点开始（深度 = 0）。
- 沿着路径往下扩展，但只允许走到 深度 $\leq d$ 。
- 到达深度 d 时，不再继续往下展开。
- 因此，所有路径都被“截断”在 d 层以内。

优点

- 避免 DFS 在无限路径里走不出来。
- 节省内存空间。

缺点

- 只能找到 深度 $\leq d$ 的解。
- 如果真正的解在更深处 ($> d$)，DLS 就会失败。

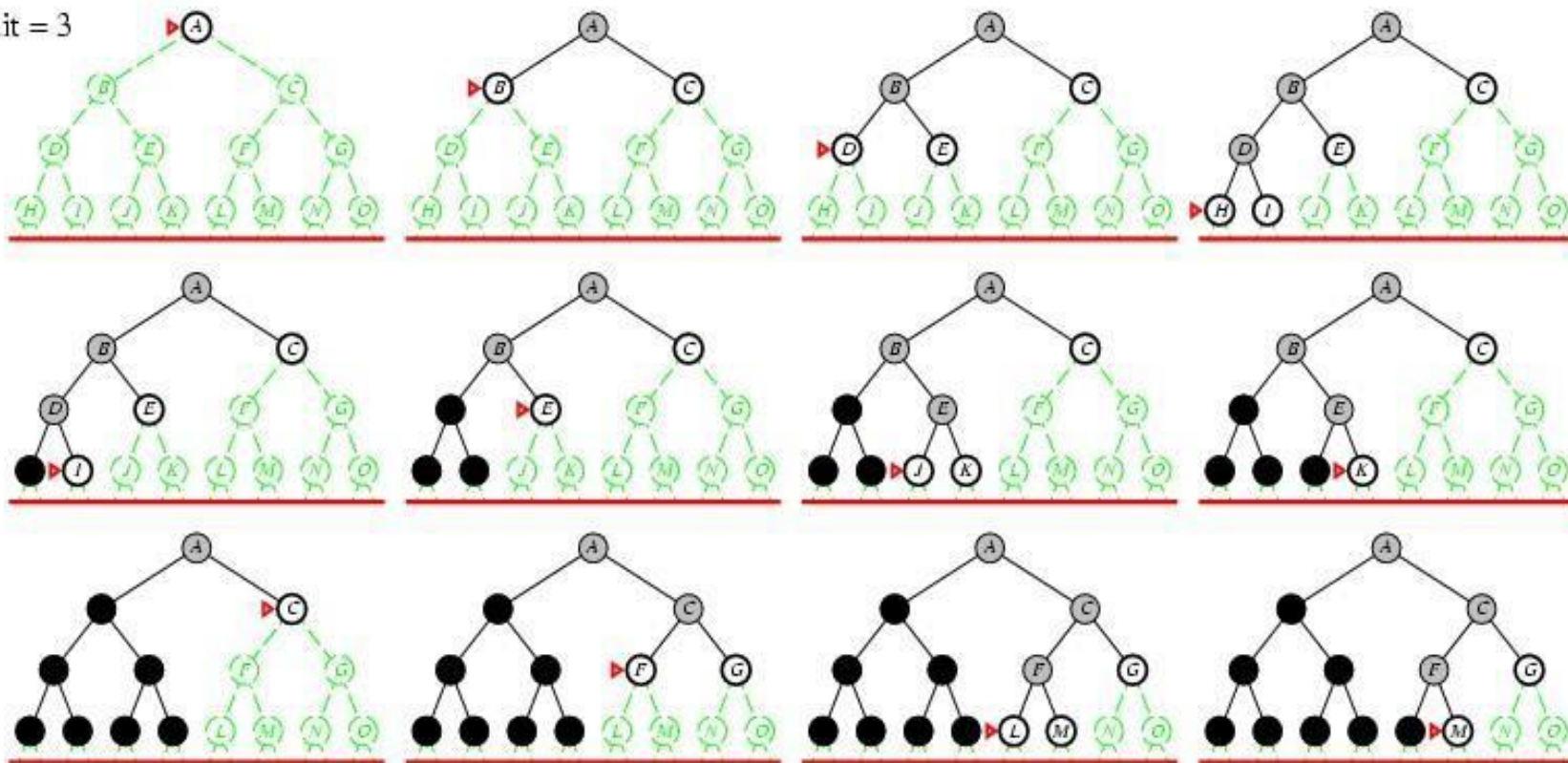
深度受限搜索

```
DLS (Frontier, Successors, Goal?) /* Call with Frontier = {<START>} */
  WHILE (Frontier not EMPTY) { n= select
    first node from Frontier
    Curr = terminal state of n
    If(Goal?(Curr)) return n

    If Depth(n) < D    //Don't add successors if Depth(n) = D!!
      Frontier= (Frontier- {n}) U Successors(Curr)
    Else
      Frontier= Frontier- {n} CutOffOccured =
      TRUE.
  }
  return FAIL
```

深度受限搜索

Limit = 3



迭代加深的深度受限搜索 (IDS, Iterative Deepening Search)

- 通过扩展深度受限搜索来解决DFS和BFS的问题
- 从深度限制 $L = 0$ 开始，我们迭代地增加深度限制，对每个深度限制执行深度受限搜索
- 如果找到解决方案，或者如果深度受限搜索失败且没有切断任何结点，则停止
 - 如果没有结点被切断，搜索将检查状态空间中的所有路径，并且发现没有解决方案→不存在解决方案。

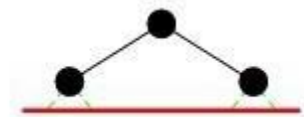
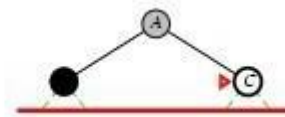
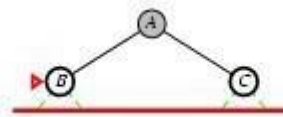
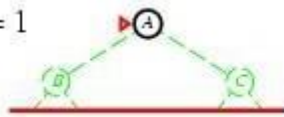
迭代加深的深度受限搜索 (IDS)

Limit = 0



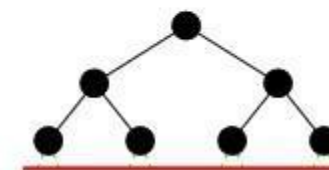
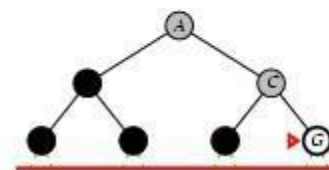
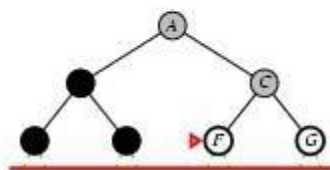
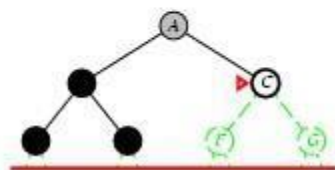
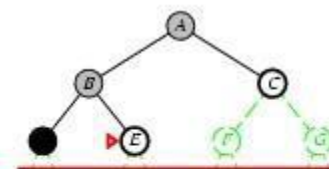
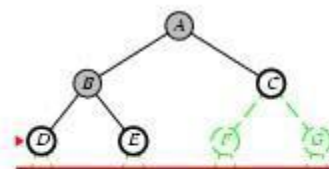
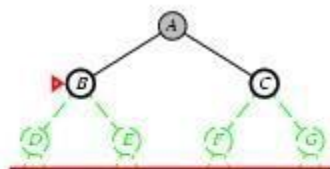
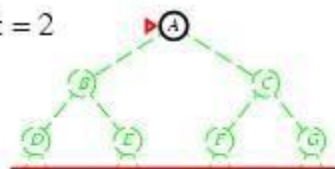
迭代加深的深度受限搜索 (IDS)

Limit = 1



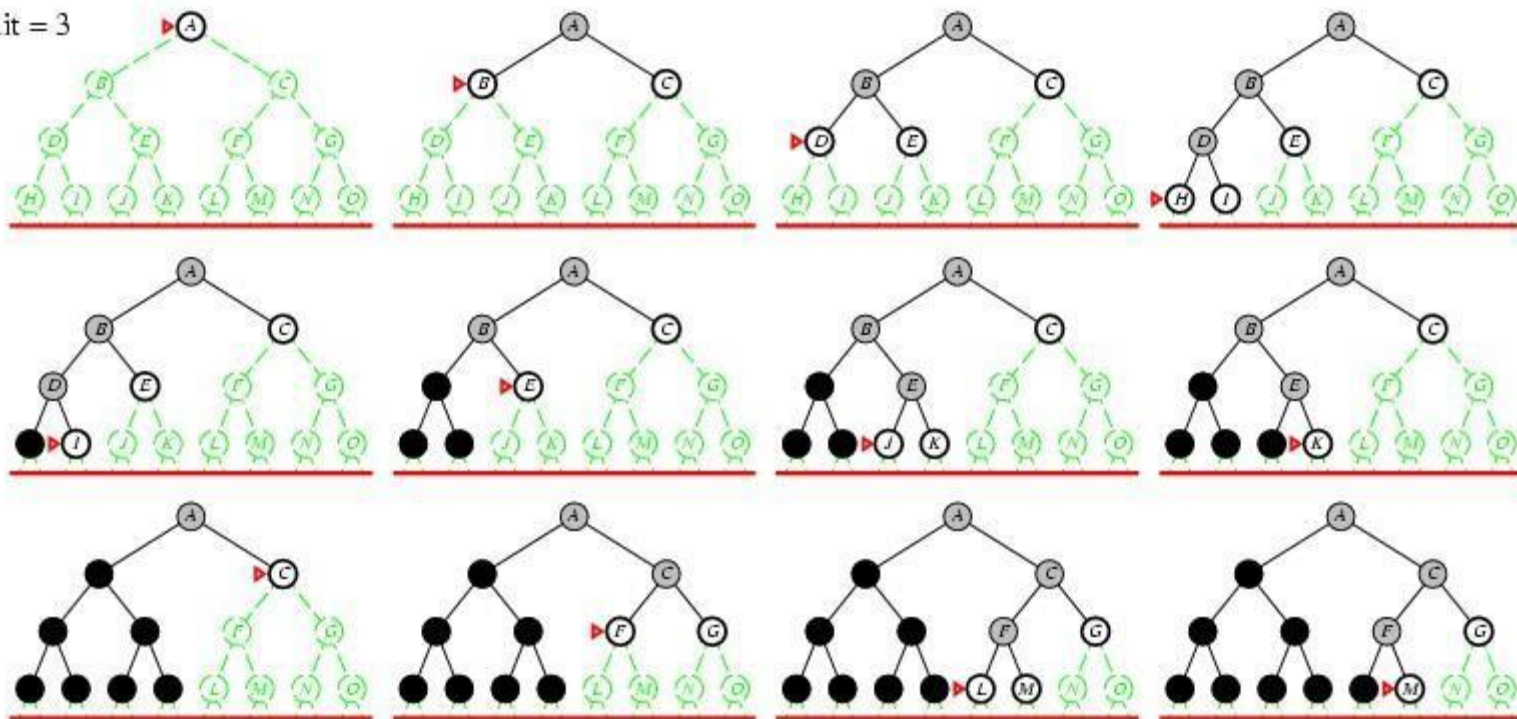
迭代加深的深度受限搜索 (IDS)

Limit = 2



迭代加深的深度受限搜索 (IDS)

Limit = 3



迭代加深的深度受限搜索 (IDS)

- 完备性？
 - 是的。如果深度限制为 d 的解存在的话
- 时间复杂度？

迭代加深的深度受限搜索 (IDS)

- 时间复杂度？

深度 0 的节点访问了 $d + 1$ 次。

深度 1 的节点访问了 d 次。

深度 2 的节点访问了 $d - 1$ 次。

...

深度 d 的节点访问 1 次。

- $(d+1)b^0 + db^1 + (d-1)b^2 + \dots + b^d = O(b^d)$

- 例如： $b=4, d=10$

- $(11)*4^0 + 10*4^1 + 9*4^2 + \dots + 4^{10} = 1,864,131$

- $4^{10} = 1,048,576$ 大多数结点位于底层

- 时间复杂度？ —— IDS 的存储方式和 DFS 完全相同，空间复杂度 $O(bd)$ 。

IDS —— 结合 BFS（完备、找到最浅解）和 DFS（省空间）优点的折中方案

BFS比IDS可以探索更多的状态!

- 时间复杂度IDS:

- $(d+1)b^0 + db^1 + (d-1)b^2 + \dots + b^d = O(b^d)$

- 时间复杂度BFS: $1 + b + b^2 + b^3 + \dots + b^d + \underline{b(b^d - 1)} = O(b^{d+1})$

当 BFS 展开第 d 层的节点时, 会生成第 $d + 1$ 层的所有子节点。即使这些子节点不会被继续展开 (因为在第 d 层已经找到了解)

- 设 $b=4, d=10$

- IDS:

- $(11)*4^0 + 10*4^1 + 9*4^2 + \dots + 4^{10} = 1,864,131$ (产生的状态数)

- BFS:

- $1 + 4 + 4^2 + \dots + 4^{10} + \underline{4(4^{10} - 1)} = 5,592,401$ (产生的状态数)

一致代价搜索 (UCS)

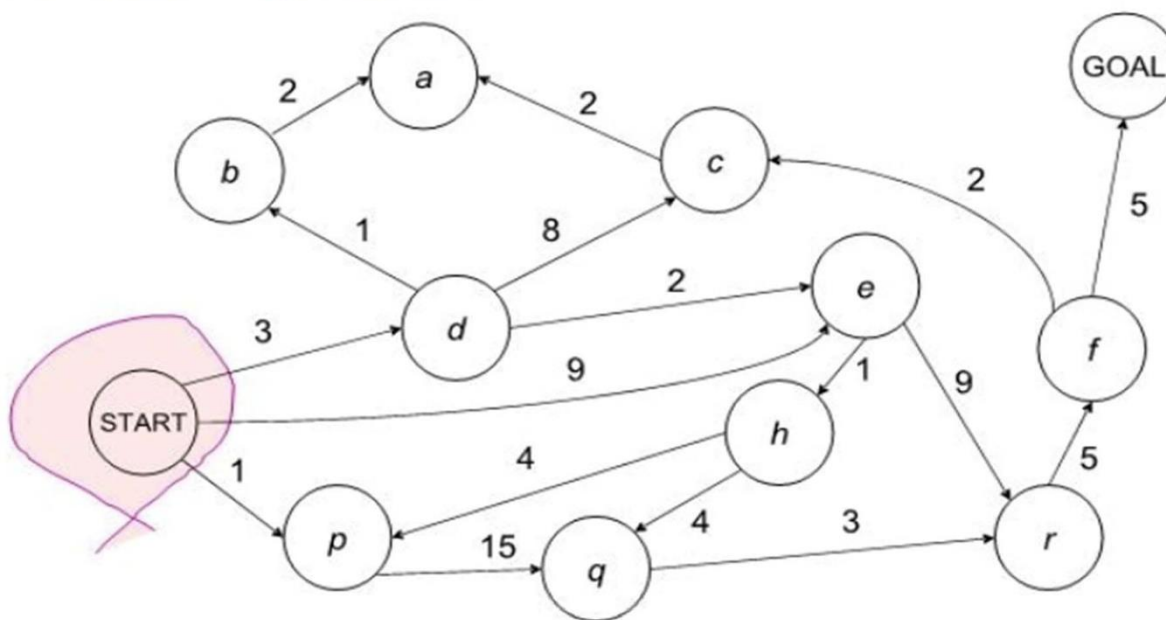
当存在路径代价时....

- UCS在边缘中要把路径代价进行升序的排序
- 始终扩展**代价最低**的节点
- 如果每个动作都具有相同代价，那么UCS等价于BFS

特性	BFS 队列 (FIFO)	优先队列 (Priority Queue, PQ)
出队顺序	按照生成顺序	按照累计代价最小
目标保证	最浅解	最小代价解
数据结构	普通队列	优先队列 (通常用最小堆实现)
示例场景	每一步代价相同	每一步代价不同

一致代价搜索 (UCS)

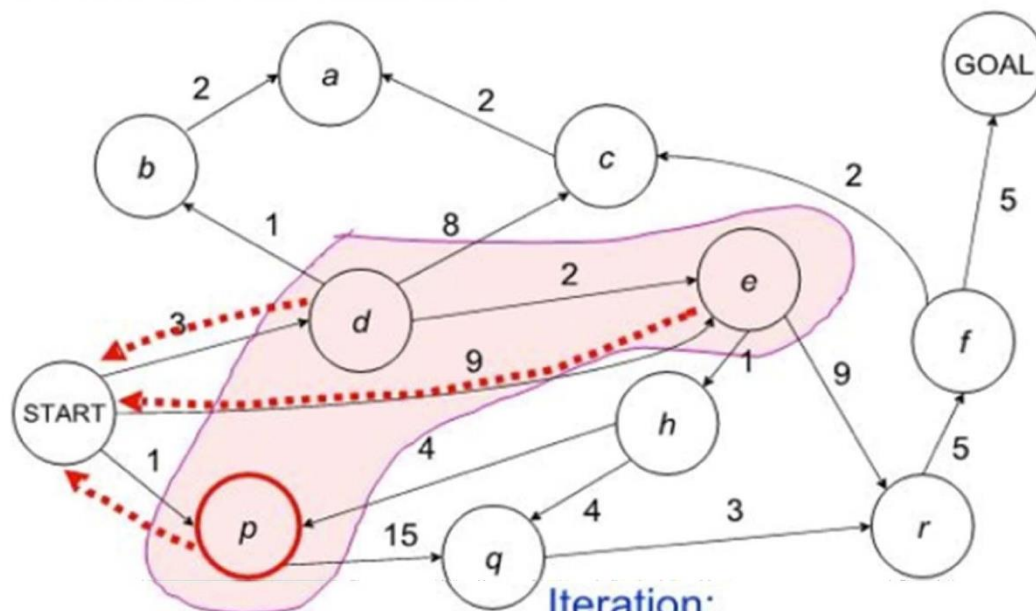
Starting UCS



Frontier = {(START,0)}

一致代价搜索 (UCS)

UCS Iterations



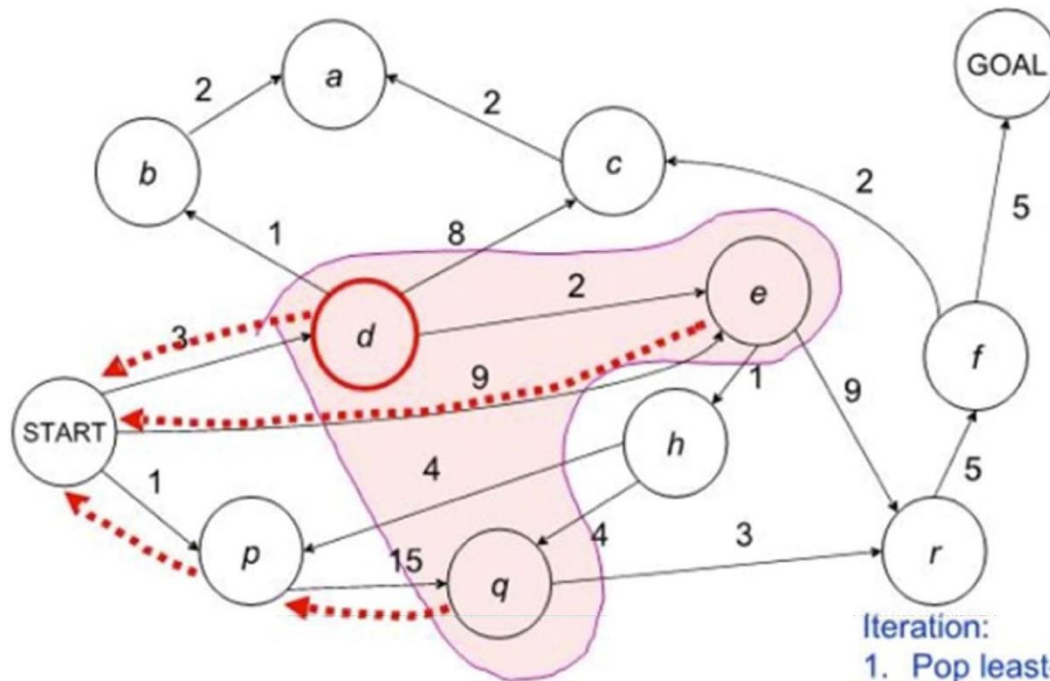
Iteration:

1. Pop least-cost state from PQ
2. Add successors

Frontier = $\{(p,1), (d,3), (e,9)\}$

一致代价搜索 (UCS)

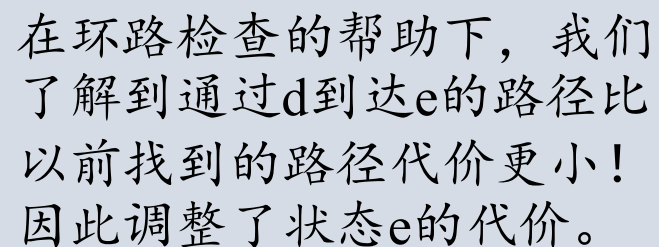
UCS Iterations



Frontier = {(d,3), (e,9), (q,16)}

- Iteration:
1. Pop least-cost state from PQ
 2. Add successors

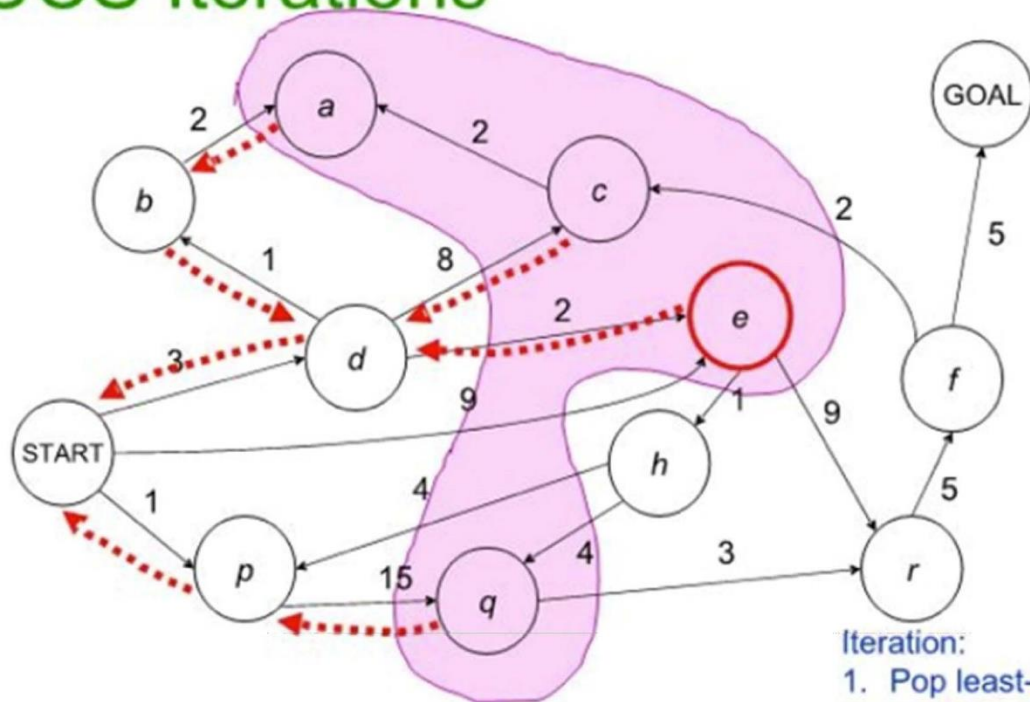
UCS Iterations



Frontier = {(b,4), (e,5), (c,11), (q,16)}

一致代价搜索 (UCS)

UCS Iterations

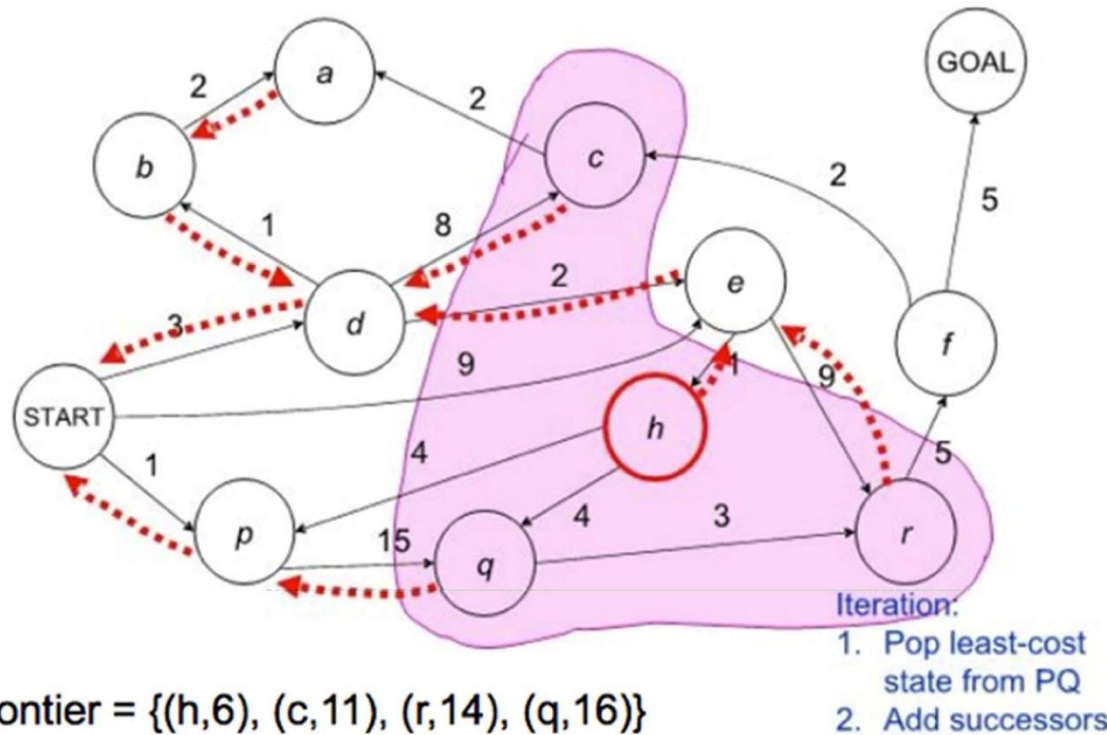


Frontier = $\{(e,5), (a,6), (c,11), (q,16)\}$

Iteration:
1. Pop least-cost state from PQ
2. Add successors

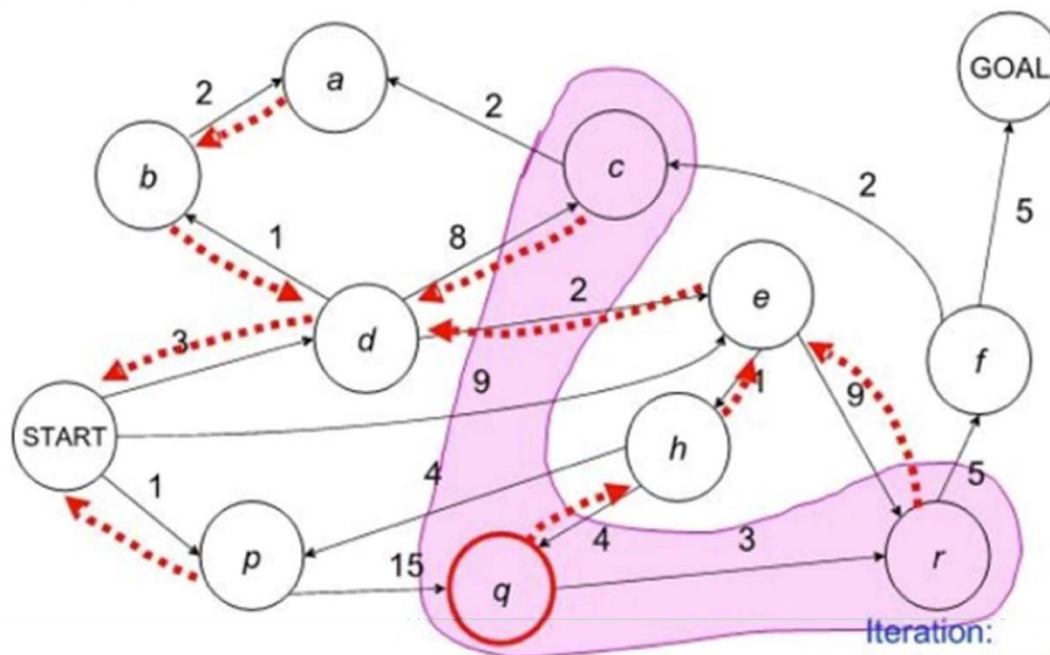
一致代价搜索 (UCS)

UCS Iterations



一致代价搜索 (UCS)

UCS Iterations

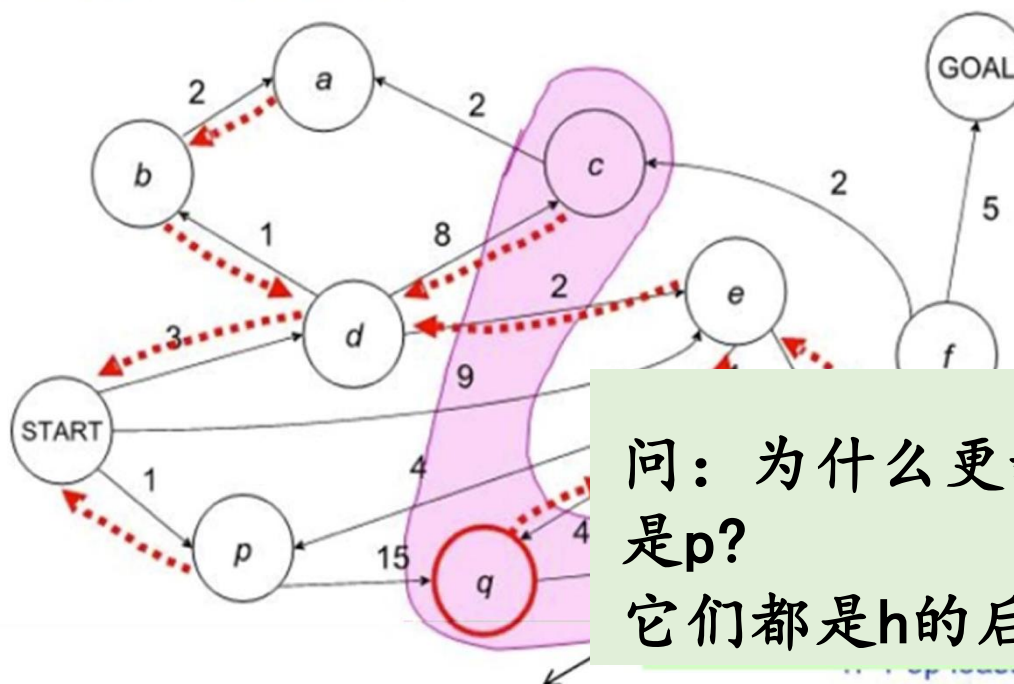


Frontier = $\{(q, 10), (c, 11), (r, 14)\}$

- Iteration:
1. Pop least-cost state from PQ
 2. Add successors

一致代价搜索 (UCS)

UCS Iterations



问：为什么更新q而不是p？

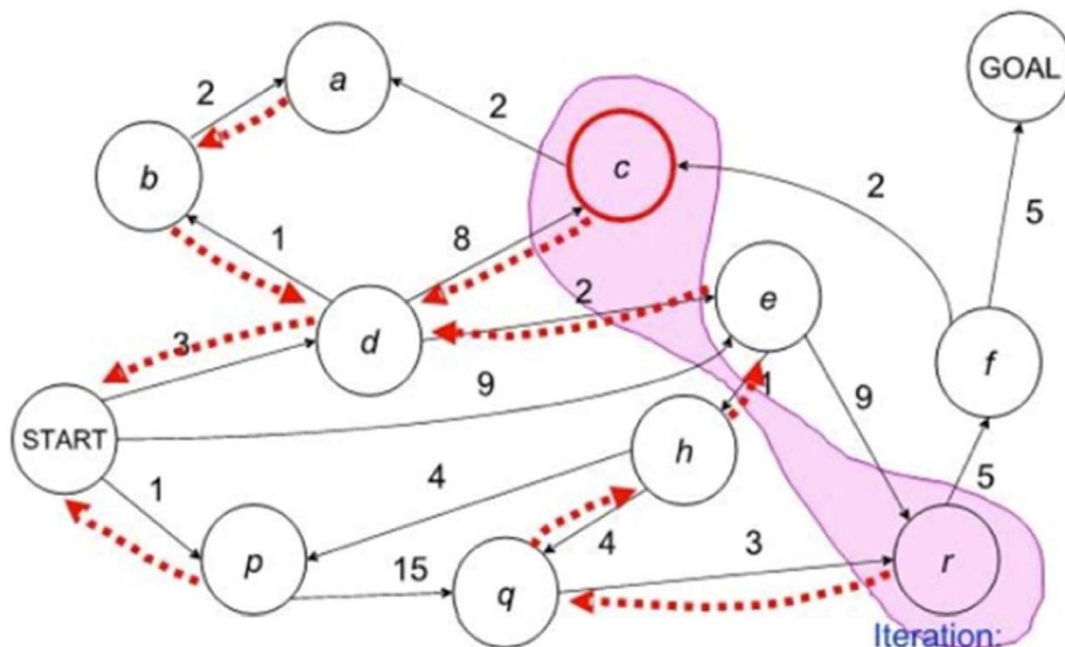
它们都是h的后继状态

Frontier = $\{(q,10), (c,11), (r,14)\}$

1. Extract best state from PQ
2. Add successors

一致代价搜索 (UCS)

UCS Iterations

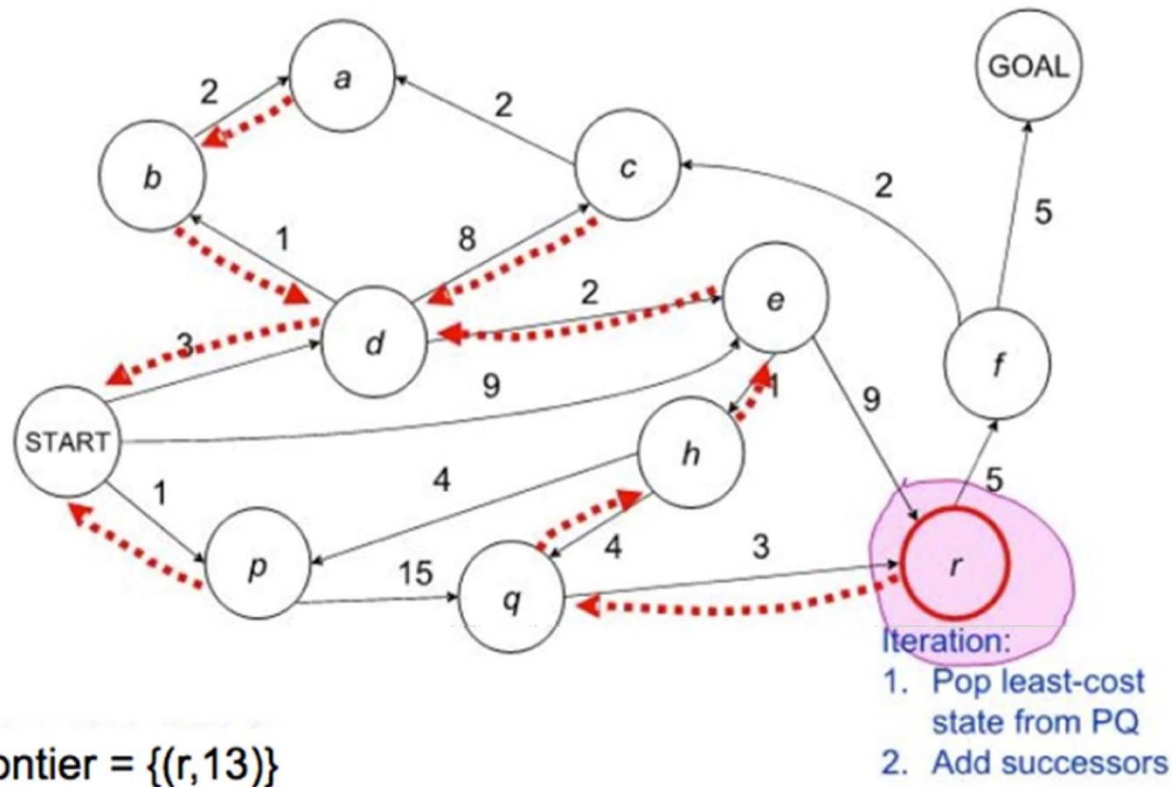


Frontier = {(c,11), (r,13)}

- Iteration:
1. Pop least-cost state from PQ
 2. Add successors

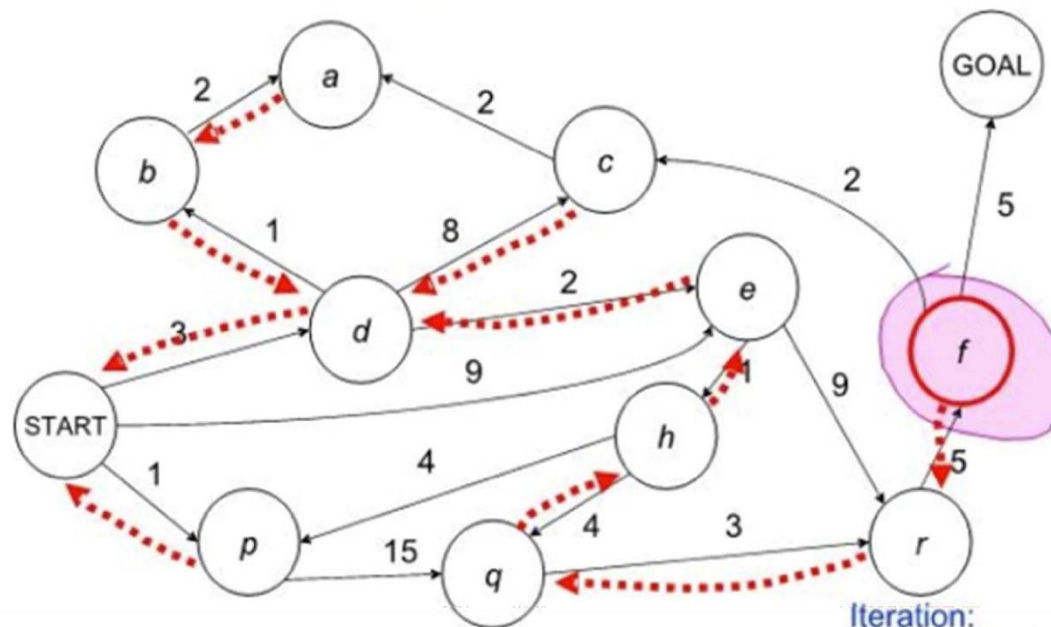
一致代价搜索 (UCS)

UCS Iterations



一致代价搜索 (UCS)

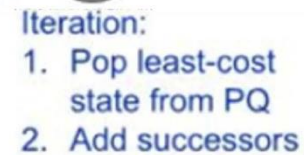
UCS Iterations



Frontier = {(f,18)}

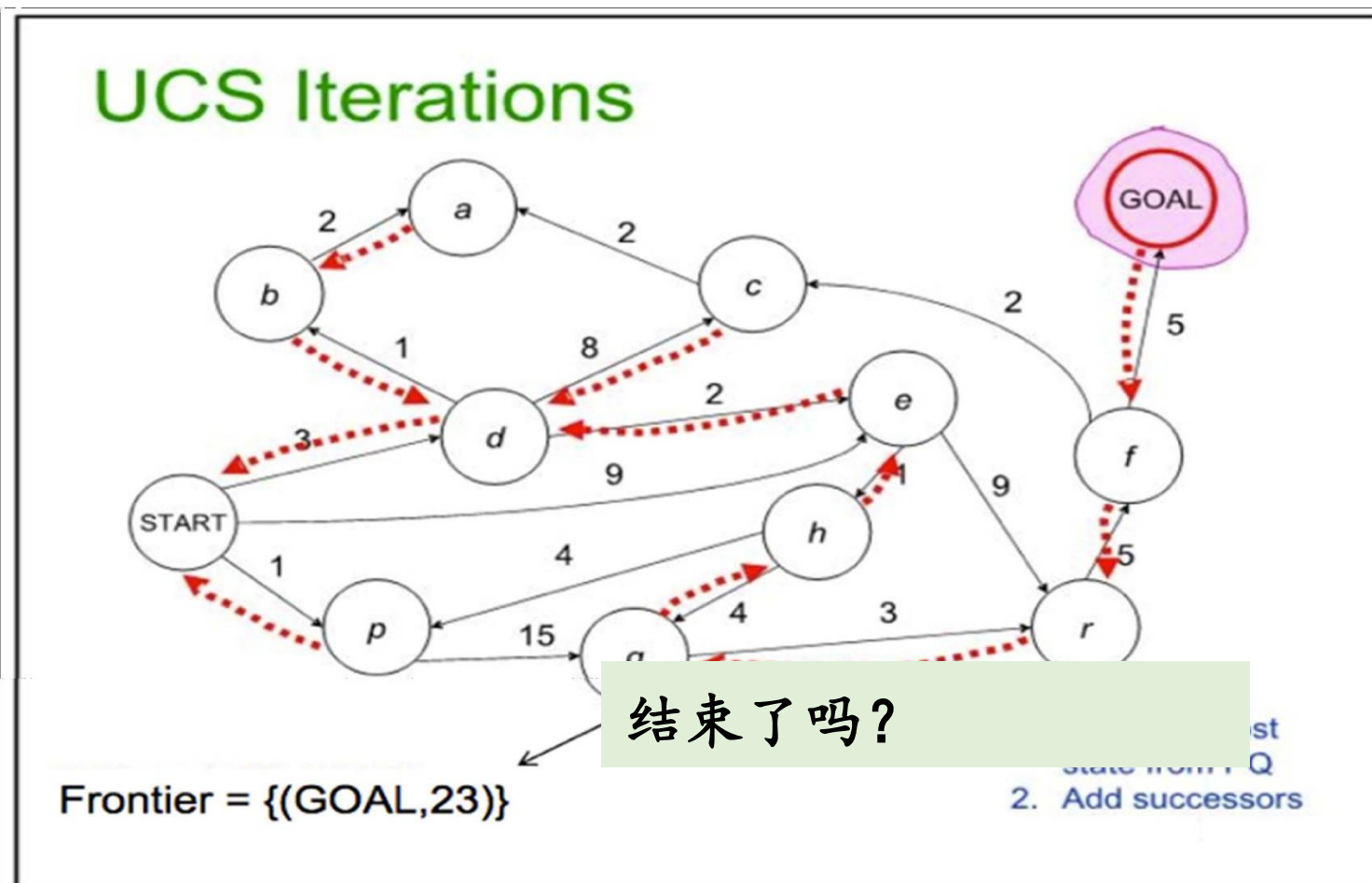
- Iteration:
1. Pop least-cost state from PQ
 2. Add successors

UCS Iterations



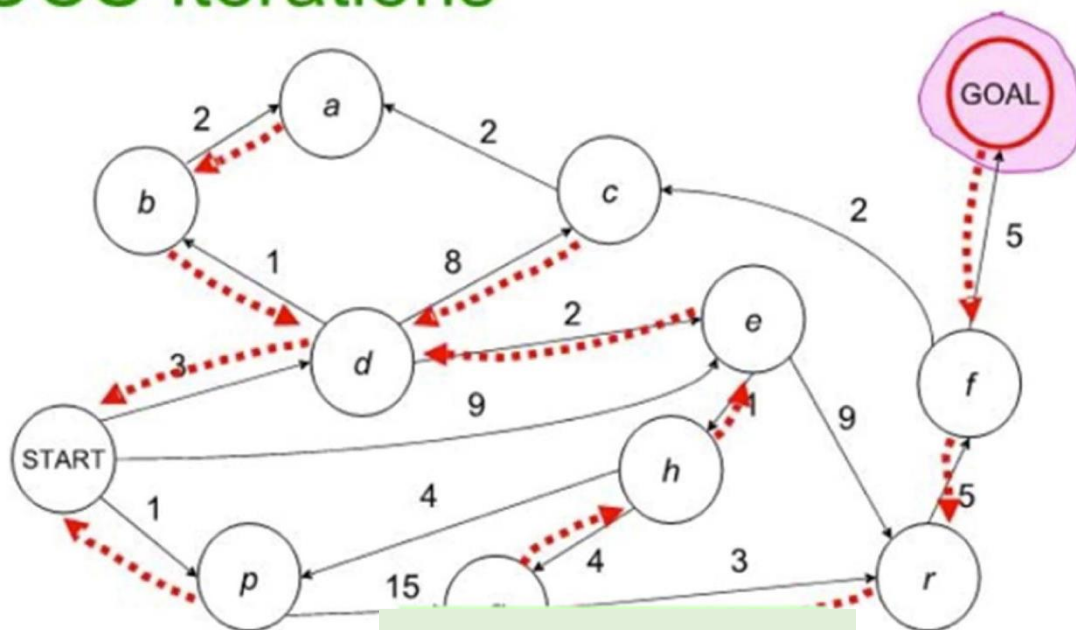
Frontier = {(GOAL,23)}

一致代价搜索 (UCS)



一致代价搜索 (UCS)

UCS Iterations



Frontier = {}

并没有，只有
边缘为空才算
结束

Iteration:

1. Pop least-cost state from PQ
2. Add successors

一致代价搜索 (UCS)

完备性？

- 每条边的代价 $\geq \epsilon > 0$ (即至少有一个正的最小代价)。
- 如果每条边代价至少是 ϵ ，那么 UCS 不会无限扩展某条零代价路径，最终一定会找到代价最小的解。

$\Rightarrow$ 在正代价图中，UCS 是完备的。

一致代价搜索 (UCS)

最优性？

UCS总是从优先队列里取出路径代价最小的节点进行扩展。

一致代价搜索 (UCS)

时间复杂度?

设:

C^* : 最优解的总代价;

ϵ : 边的最小代价。

在最坏情况下, UCS会扩展所有代价 $\leq C^*$ 的节点。

节点数大约是:

$$O\left(b^{\frac{C^*}{\epsilon}}\right),$$

其中 b 是分支因子, 一条路径的最长步数不会超过 $\frac{C^*}{\epsilon}$

直观理解: 搜索范围取决于“总代价半径”, 而不仅仅是深度 d 。

一致代价搜索 (UCS)

空间复杂度？

UCS 必须把所有frontier节点（优先队列）存起来，因为随时可能要从中取出最小代价的节点。

所以空间复杂度 $\approx$ 时间复杂度：

UCS 的最大代价是存储整个边界，和 BFS 一样，空间开销非常大。

总结

算法	核心特点	优势	应用
BFS (广度优先)	用队列，逐层展开， 保证找到最浅解	找最少步数的解 (所有边代价相同)	- 社交网络最短关系链（如 Facebook/LinkedIn 好友推荐） - 迷宫/棋盘最少步数解
DFS (深度优先)	用栈或递归，沿一 条路径走到底再回 溯，空间省	快速找到一个解， 不保证最浅解	程序分析/推理链搜索
DLS (深度限制)	在 DFS 基础上设定 深度上限	避免 DFS 掉进无限 深度	- 爬虫设深度限制，防止死循环
IDS (迭代加深)	多次限制深度的 DFS，结合 BFS 的 最优性和 DFS 的省 空间	在不知道解深度时， 既保证找到最浅解， 又节省空间	- 路径规划（在空间有限时比 BFS 更好） - 机器人探索未知环境
UCS (统一代价)	用优先队列，每次 扩展总代价最小的 节点	找总代价最小的解， 适合边代价不等	- 地图/GPS 路径规划（高德/Google Maps，最短路径） - 机器人运动规划（能量消耗最小路径）

总结：搜索算法

